

What Reflection Is Told, Not Where It Sits: A Controlled Study of Self-Reflection in Multi-Turn Tool Calling

Mirodil Kamilov

SS 2026

Abstract

Self-reflection is a widely adopted way to improve LLM agents: an agent whose episode fails writes down what went wrong, and that lesson is carried into a fresh attempt. Its largest reported gains come from settings with rich feedback, such as program synthesis. This report asks whether it survives a harder setting — BFCL v3 multi-turn, where an agent sustains a stateful conversation across several turns of API calls and the feedback a reflection step may legitimately receive is far thinner. I evaluate Reflexion against a *sham* reflection, matched in form and length to a real one but carrying only general advice, never a lesson from the failure. The sham performed slightly better (-1.5 pp for reflection, 95% CI $[-5.5, +2.5]$). Supplying the reflection step with a substantially richer input — a record of how the agent’s own actions changed the environment on each turn — also changed nothing ($+0.0$ pp). Reflection is nevertheless not inert. Across six conditions of increasing signal quality, only 28 of 115 failed tasks are recovered by *any* retry, and every condition recovers 15–18 of those same 28: reflection alters *which* failures are recovered rather than *how many*.

1 Introduction

An LLM agent solves a task by alternating between reasoning and acting: it inspects its situation, calls a tool, reads the result, and repeats. Two influential proposals add an explicit reflective step to that loop. ReAct [7] has the model write a short thought before each action. Reflexion [6] waits until an episode has failed, has the model write a verbal lesson explaining the failure, and carries that lesson into a fresh attempt. Between them sits a third position — reflecting after each individual tool result, to decide whether the last action should be revised, which the REBACT [8] line of work explores.

I set out to compare these three positions directly. The plan was to build five architectures — a plain function-calling baseline, ReAct, episodic Reflexion, a Reflexion variant with a persistent cross-task memory, and REBACT — run them over one frozen set of tasks, and answer a single question: *where in the reason-act loop does reflection actually help?*

1.1 Why this benchmark, and why the plan did not survive it

The original ReAct and Reflexion results were obtained largely on ALFWorld, HotpotQA and WebShop. Those tasks are demanding, and some are long — an ALFWorld episode can

run for tens of steps — but they share a narrow notion of what the agent must get right: each episode carries a single instruction, chosen from a small action vocabulary, and is scored on whether the goal was reached. HotpotQA, for instance, gives the agent three actions.

BFCL v3 [4] is however a *multi-turn* and *multi-step* benchmark: an episode consists of several realistic user–agent interactions, and each turn may require chaining several calls to satisfy one request. A task in my subset exposes a median of 28 callable functions (range 17–39). The agent is graded on both the resulting environment state and the minimum set of calls the task requires. A further part of the difficulty is about not acting at all: two categories test abstention, which neither earlier benchmark does at all — whether the agent asks for an argument the user omitted (`miss_param`), and whether it recognises that no available tool fits (`miss_func`).

A final difference bears on reflection directly. Reflexion’s largest gains come from program synthesis, where the reflection step reads a failing test suite — a signal close to a diagnosis in itself. BFCL offers no test output and no partial credit, only a verdict, so a reflection step must infer its own diagnosis.

Three findings, in order, made the original comparison untenable.

First, the measurement floor turned out to be high. The model used here, `qwen3-next-80b-a3b-instruct` [5], is a mixture-of-experts model served over a shared endpoint, and at `temperature=0` its outputs are still not reproducible. Running the identical baseline twice over the same 200 tasks produced different verdicts on 15 of them. That places a floor on what the experiment can resolve: any true effect below roughly 5 percentage points (pp) is indistinguishable from noise, and the effects under investigation are plausibly that size.

Second, the ReAct condition produced a null for a reason specific to modern models. ReAct was proposed for base models that do not spontaneously verbalise a plan, so instructing them to do so supplies something genuinely absent. The instruction-tuned model used here already writes free-text reasoning before 80% of its tool calls without any prompting. The scaffold therefore had very little left to contribute, and the pass rate did not move. The result characterises the model rather than the method.

Third, and decisively, a control arm behaved unexpectedly. To separate the value of a reflection’s *content* from the value of simply attempting the task again, I added an arm in which the model’s own lesson was replaced by a fixed paragraph, matched in form and length to a real reflection but saying nothing about the actual failure — a *sham*. The expectation was that this arm would fall clearly behind genuine reflection. It did not; it scored marginally higher. Reducing the sham’s lesson further, to a single bland sentence, changed nothing again. Whatever was producing the gain, it was not the content of the reflection.

That observation is not isolated: Kapoor et al. [2] find that on HumanEval a bare retry baseline — re-invoking the model at temperature zero whenever the tests fail — matches the best published agent architectures, precisely because such models are not deterministic even at temperature zero. Their comparison runs between architectures; this report isolates the same effect within one, by holding the retry fixed and varying only what the reflection is told.

1.2 The revised question

If reflection was contributing nothing, one natural explanation was that it had too little to work with: the agent was told only *which* simulated API ended in the wrong state, or on *which* turn it failed. I therefore built a further arm that additionally shows the reflection step a record of what its own actions changed in the environment on each turn — the difference between the environment state before and after every turn of its failed attempt. This is more information, it is cheap, and it cannot leak the expected answer. It too changed nothing.

The question consequently shifted from *where* reflection sits in the loop to *what it is told*:

Does the information content of the reflection signal change *how many* failures an agent recovers, or only *which ones*?

The answer is: only which ones. This seminar project’s contribution is adding the evaluation — six-condition ladder of steadily richer reflection input over one frozen task set and a sham control that separates reflection from retrying (Section 4.3); evidence that recoverable failures form a small fixed pool, so reflection reallocates rather than enlarges the set of tasks solved (Sections 4.4–4.5); and a hand-coded audit of the reflections that explains why a richer signal did not help (Section 5).

2 Background and setup

How BFCL grades. A task supplies the model with a set of simulated APIs — a file system, a trading bot, a travel booking service — and a conversation in which the user makes one request per turn. After each turn, two gates must both pass. The *state gate* replays the model’s calls and requires every public attribute of every involved simulator to equal the ground truth’s. The *response gate* requires the ground truth’s tool outputs to be a subset of the model’s.

This has a consequence that shapes several results below. Extra *read-only* calls are free: an agent may call `ls` or `get_balance` as often as it likes, since neither gate penalises additional output or unchanged state. Extra *state-changing* calls are not — an agent that satisfies every request but also alters something the ground truth left alone fails the state gate (Section 5 gives a case).

The four categories. Besides `miss_param` and `miss_func` above, base is ordinary multi-turn tool use and `long_context` pads the environment so the relevant details are buried. The two abstention categories pull in opposite directions: `miss_param` penalises an agent that acts when it should have asked, while `miss_func` penalises one that hesitates when it should have acted. An agent that shifts its general willingness to act will therefore gain in one category and lose in the other, a pattern that recurs throughout the results.

Setup. One model throughout, `qwen3-next-80b-a3b-instruct`, served over a university endpoint. A fixed subset of 200 tasks (50 per category, stratified by which APIs a task involves, frozen to a committed file and shared by every arm). `temperature=0`, one run per task, an 8192-token cap that never binds, and a 30-step-per-turn guard that fires on 1–4 tasks per arm. All code, prompts, trajectory logs and per-task results are public [1].

Three fairness rules hold everywhere, because breaking any one of them would make the comparison meaningless. Every arm emits *actions* through the same native function-calling channel, so no arm is penalised by a text-parsing failure the others do not face; only the

added *thinking* is free text. Every arm is zero-shot. Every arm shares one identical task-instruction core in its system prompt, and differs only by its reasoning or reflection scaffold (Appendices A.1 and A.2).

3 Method

Six conditions run on the same 200 tasks. **Baseline** (plain function calling) and **ReAct** do not retry. The other four share a single retry loop and differ only in what fills its reflection slot: a fixed sham paragraph (**Sham**), a shorter one with the advice removed (**Sham-lite**) (Appendix A.4), the model’s own lesson about the failure (**Reflexion**), or that lesson plus a record of what the agent’s own actions changed (**Reflexion+State**). Table 2 states each precisely.

3.1 The shared retry loop

Reflexion retries up to $k = 3$ times. Retries are **fresh-episode**: the failed conversation is discarded, the simulated environment is reset to its initial configuration, and the retry is a new conversation that opens with a short user message before turn 0 saying that a previous, separate attempt failed, carrying the failure signal and the lesson (Appendix A.5). Discarding the transcript is what makes the sham control meaningful: were the failed conversation left in context, an agent given an empty lesson could re-read it and diagnose the failure itself, silently acquiring the information the sham is supposed to lack.

Attempt 1 of every retry arm reuses the recorded baseline run rather than executing a fresh one. A task the baseline passed is carried through as a pass, with no reflection written and no retry run; only the 115 tasks it failed enter the retry loop. Reflexion@1 is therefore the baseline by construction, and all four retry arms retry an identical failure set.

The failure signal is an oracle: the grader tells the agent that it failed. Baseline and ReAct never receive it, so any comparison between them and a retry arm carries that advantage; all four retry arms receive the identical signal, so it cancels in the contrasts this report relies on. It is also **sanitised**, because the grader’s raw output contains the expected answer — the agent is told only which API class mismatched, or on which turn, and never any value (Appendix A.6).

3.2 Why a sham control is necessary

Because the endpoint is not reproducible, a second attempt can succeed even when the lesson contributes nothing: 7.8% of failed tasks pass on an identical re-run. The gap from @1 to @ k therefore confounds two quantities:

$$\text{@1} \rightarrow \text{@}k = \underbrace{\text{value of the lesson}}_{\text{the quantity of interest}} + \underbrace{\text{effect of re-attempting}}_{\text{confound}}$$

One measurement cannot separate two unknowns. The remedy is a control arm identical in every way except that the lesson slot holds a fixed constant — a *sham* reflection, with the same three headings and comparable length, written by no one (Appendix A.4). Then **Reflexion@ k – Sham@ k** isolates the reflection content, and **Sham@ k – @1** absorbs the luck and the framing. **Sham-lite** bounds how much of that comes from the advice rather than from retrying.

3.3 The state signal

The contribution arm, **Reflexion+State**, is identical to Reflexion except that the reflection call also sees the agent’s **own per-turn state timeline**: what each turn changed, relative to the turn before (Appendices A.3 and A.7).¹

This is not oracle-derived, which is what keeps it within the leak constraint. The agent’s own recorded calls are replayed from the task’s initial configuration, so nothing about the expected answer enters. This added timeline block is produced once per reflection call, 218 times over the run: a genuine per-turn timeline in 175 cases; in 28, a statement that the agent changed nothing — itself diagnostic, since failing to act is the characteristic error in the abstention categories; and in 15, all `long_context`, a placeholder, because the timeline exceeded a 3500-character cap set to stop a large filesystem swamping the prompt.

3.4 Analysis

All arms share the frozen subset, so every comparison is **paired per task**. I report McNemar’s exact test [3], which discards the tasks on which two arms agree and asks whether the remaining disagreements are too one-sided to be attributed to chance, plus 20,000-sample bootstrap confidence intervals over tasks.

The **minimum detectable effect** follows from the noise: with roughly 22 disagreements per comparison, significance needs about a 16–6 split, which is a true effect of ≥ 5 pp pooled and 8–10 pp within a single 50-task category. I state this up front because most results below are smaller than that. Per category, results are descriptive only.

4 Results

4.1 The noise floor, and how much of it is the benchmark

Two identical baseline runs disagree on **15 of 200 tasks**. Of the 115 tasks that failed the first run, 9 (7.8%) pass on the second by chance alone. This is the ruler for everything else.

I hand-labelled all 15 flips against the ground truth. Ten are genuine model errors. **The remaining five are not instability at all: the agent’s behaviour was defensible and the grader rejected it.** Two examples show their character. In `base_37` the agent wrote a file with `echo`, which creates the file when it is absent; the ground truth first called `touch` and then `echo`, both achieve the requested effect, but only one satisfies the response gate. The final state is identical and the task fails anyway. In `miss_param_119` the agent verified that a user was logged in before closing their support ticket — sound practice in any real system — but `close_ticket` requires no login in the simulator, and the extra check left state the ground truth never produced. Table 1 lists all five with their mechanisms.

One category is weaker than it looks. In `miss_func`, roughly half the tasks never call the revealed tool at all (24/50 for the baseline, 21/50 for ReAct). Reading the transcripts, the model denies the tool exists — “I don’t see any new tools added” — while the request verifiably contained it. That is a failure of *perception* of the tool list, which happens before any reasoning loop could help. Expect reflection to be flat here, and it largely is.

¹Arms are named `baseline`, `react`, `reflexion`, `blind_retry`, `blind_retry_lite` and `richer_reflexion_turnwise` in the code and result files.

Table 1: The five benchmark artefacts among the 15 run-to-run flips. These are grading brittleness, not model stochasticity.

Task	Mechanism
base_117	Solved via a synonymous tool the ground truth did not choose
base_140	Ground truth demands exact free text the request underdetermines
base_37	Correct final state; skipped a redundant GT call returning <code>None</code>
miss_param_119	Real-world-sound auth check penalised; the tool needs no login
miss_param_145	Two symmetric login tools return contradictory status

Table 2: The six conditions, ordered by how much the retry is told. The lower four are the Reflexion experiments; Baseline and ReAct do not retry and are shown for reference. pass@3 is out of 200 tasks.

Arm	What it adds	pass@3
Baseline	native function calling, no scaffold	85
ReAct	a thought before each action	86
Sham-lite	retry + sanitised signal + a bland lesson	102
Sham	+ generic corrective advice	103
Reflexion	+ a lesson written from the failure	100
Reflexion+State	+ the agent’s own state timeline	100

4.2 ReAct: a manipulation with nowhere to go

ReAct scores 86/200 against the baseline’s 85, a difference of +0.5 pp (12 wins, 11 losses, $p = 1.00$) — indistinguishable from the two baseline runs’ disagreement with each other. The manipulation check explains why. With no scaffold at all, the baseline already writes free-text reasoning before **80.2%** of its tool calls. ReAct moves that to **83.0%** (+3.1 pp, CI $[-0.3, +6.4]$, sign test $p = 0.033$) and adds about 20 characters per action ($p = 0.003$). The literal **Thought:** label appears on only 21.6% of action steps. So the scaffold did shift behaviour slightly, but the behaviour it targets was already near ceiling, and the pass rate did not move.

To be precise, what was tested is zero-shot ReAct-style thought injection over native function calling — not ReAct as published, which was demonstrated mainly on a pre-trained (not instruction-tuned) model, with few-shot exemplars and actions parsed out of text — two differences forced by the fairness rules above, which require every arm to be zero-shot and to act through the native function-calling channel.

4.3 The ladder: only the first step matters

Table 2 summarises the Reflexion experiments: the four retry arms, together with the two non-retrying conditions they are measured against. Signal content increases at every rung and the score does not. The four retry arms span three tasks (100 to 103) — the same spread as two identical runs of the Baseline, which scored 85 and 88.

The one large effect is the retry itself: Sham reaches 103 against the 85 every arm shares at @1 (first-attempt — seeded from Baseline), a gain of **+9.0 pp** (18 wins, 0 losses, $p < 10^{-4}$) — necessarily one-sided, since a retry fires only on a failure and inherits every pass. After that, nothing moves.

- **Real reflection loses to the sham.** Reflexion – Sham is **−1.5 pp**, CI $[-5.5, +2.5]$, 6 wins to 9 losses, $p = 0.607$.

Table 3: Pass counts per category (of 50). The pooled null hides opposite-signed category effects. Descriptive only: per category the detectable effect is 8–10 pp.

Category	pass@3			difference	
	Sham	Reflexion	+State	Refl.–Sham	+State–Refl.
base	38	37	37	−2 pp	±0 pp
miss_func	10	14	10	+ 8 pp	− 8 pp
miss_param	20	15	19	− 10 pp	+ 8 pp
long_context	35	34	34	−2 pp	±0 pp

- **The sham’s advice is inert.** Sham − Sham-lite is +0.5 pp (2 to 1, $p = 1.00$), with three of four categories identical task-for-task. The sham arms flip 12.2% of failures on the first retry against 7.8% for a plain re-run, so that 4.4 pp belongs to the retry framing and the sanitised signal, not to advice.
- **The richer signal changes nothing.** Reflexion+State − Reflexion is +**0.0** pp, CI $[-3.5, +3.5]$, 7 to 7, $p = 1.00$.

This is the result the control bought. Without a sham arm, the honest-looking headline would have been “Reflexion +7.5 pp, $p < 10^{-4}$ ”, and every point of it would have been retry luck.

4.4 Which failures, not how many

The pooled zero is not a flat null. It is two opposite effects cancelling (Table 3).

Reflection reliably moves one thing: the agent’s threshold for *acting* versus *asking*, since its lessons converge on generic rules about when to act and when to check with the user. In `miss_func`, where the dominant failure is under-acting, that push helps (+8 pp). In `miss_param`, which punishes hesitancy, the same push over-corrects into caution and hurts (−10 pp); lessons such as “always confirm the user ID before acting” are exactly wrong there.

Adding the state signal moves the threshold back, in both directions at once: `miss_param` recovers (15 → 19, back to the sham’s level) at the exact cost of `miss_func` (14 → 10, also back to the sham’s level). The behaviour confirms it. Tool calls per retry in `miss_param` run 8.00 for Reflexion, 8.87 for Reflexion+State and 9.33 for the Sham — the arms are ordered by how willing they are to act. So whether reflection helps is a property of the task category, not of the reflection. Averaged over four categories it cancels.

4.5 The ceiling: a small fixed pool

There is a simpler reason all four retry arms land within three tasks of each other: re-attempting does not act uniformly on the 115 failures. A small pool of borderline tasks flips readily and the large majority are locked. Across five independent re-roll conditions only **28 of the 115 failures are ever recovered by anything**, and each arm recovers 15–18 of those same 28 (Sham 18, Sham-lite 17, Reflexion 15, Reflexion+State 15). The other 87 never flip for anyone; they are the `miss_func` tool denials and the grading artefacts of Section 4.1. Reflection therefore reallocates within a fixed pool rather than enlarging it: Reflexion and the Sham overlap on only 38% of the tasks they convert, and the split is category-structured.

Table 4: Hand-coded quality of the 14 discordant attempt-1 reflection pairs. No bucket differs by more than two tasks.

Arm	Diagnosis			Lesson		
	correct	partial	wrong	correct	partial	wrong
Reflexion	6	3	5	6	2	6
Reflexion+State	5	4	5	5	4	5

4.6 Cost

The retry arms cost about $2.1\times$ the Baseline’s input tokens (29.5M against 13.8M), and that multiple is almost entirely the extra attempts. Within Reflexion the reflection mechanism accounts for roughly 5% of the tokens — 2% to generate the lessons, and about 3% more to carry them, since a lesson sits in the retry’s opening message and is re-sent with every call in that attempt. The state timeline adds a further 0.1%. So the expensive component of Reflexion is the retry, which worked, and the cheap component is the reflection, which did not.

5 Why the richer signal did not help

5.1 It changed salience, not information

The state timeline is largely something the agent already had. Across all 175 real timelines (654 attribute lines), **47.6% of the values appear verbatim in the transcript the reflection step is already reading**, and most of the rest is derivable from tool results in that same transcript. The timeline block restructures and aggregates; it does not inform. This is a lexical test and should be read as one, but it sets the scope of the null: what remains untested is a signal genuinely *new* to the agent, such as an external validator or an environment probe it could not otherwise observe.

5.2 A hand-coded audit of the reflections

The two reflection arms disagree on 14 tasks. Their attempt-1 reflections form an unusually clean comparison, because the shared seeding makes the transcript, the signal, the prompt template and the temperature identical, leaving the state block as the only difference. I read all 28 against the ground truth and coded them on two axes: whether the *diagnosis* names the real cause, and whether following the *lesson* would help (Table 4). The two arms are level, with Reflexion if anything marginally ahead on diagnosis. The block changed the conclusion on 6 of the 14 tasks, but symmetrically — improving the diagnosis twice and degrading it twice — and it went unused about as often as it helped: in **base_94** the agent released a parking brake the ground truth left engaged, and the block lists that exact change while the lesson blames tyre pressure instead.

5.3 Two tasks that show the whole mechanism

Where it worked. In **miss_param_32** the ground truth requires writing 2.0 into a file. Both arms diagnose the failure correctly: the agent wrote an explanatory sentence into the file instead of the bare number. They differ on the fix. Reflexion *guessed* the format — “output only the precise numeric value (e.g. 2.0000)” — wrote 2.0000 on its retry, failed again, and doubled down. It lost because of its own lesson. Reflexion+State was shown the file it had actually written and prescribed “write only the raw number (e.g. 2.0)”. It passed.

Note what the block contributed: not understanding, which both arms had, but a concrete anchor that stopped the fix drifting into invention.

Where it backfired. In `miss_param_169` Reflexion diagnosed the failure exactly right — the user had not yet given the travel date or class, so the agent should have asked. Reflexion+State did not. Its state block showed that its own booking attempt had changed a credit-card record, and it concluded that “the system required `client_id`, `client_secret`, and other authentication details”. Those parameters are real. They belong to `authenticate_travel`, one of the 27 tools the agent held. But booking does not need them — only the `access_token` the user had already supplied. So the block did not make the model invent a requirement. It prioritised the wrong tool, and the lesson sent the retry hunting for credentials instead of asking for the missing details. Same mechanism, opposite directions — that is what a +0.0 pp result is made of.

6 Limitations

One model. All results are specific to `qwen3-next-80b-a3b-instruct`; the ReAct saturation finding in particular is a property of an instruction-tuned model that already narrates.

Two of three loop positions. I test before-acting and after-episode. Per-step reflection was cut for time, and it is the most promising untested position precisely because of this result: episodic reflection shifts the act/ask threshold *globally*, which is why opposite category effects cancel, whereas a per-step variant could apply the correction locally.

No pass@ k figure is deployment-realistic. A retry fires only when the grader reports a failure, and a deployed agent has no such signal. It also restarts from a clean environment: an agent that has already booked a flight or deleted a file cannot take that back. The setup is realistic as a data-generation procedure rather than as an architecture. A sandbox supplies exactly what deployment lacks — a failure oracle and an undo — so an agent can attempt a task repeatedly and the trajectories that pass can be used to train the model. Yao et al. apply the same generate-filter-train pattern, fine-tuning on 3,000 ReAct trajectories selected for task success [7].

The richer signal manipulated salience, not information (Section 5). The state timeline is close to everything BFCL permits without leaking the answer: it exposes no test suite, no validator, leaving only the transcript, a sanitised verdict and the agent’s own effect on the environment. A setting offering more — the stack traces of code generation, or an environment the agent may probe — would give it information it could not otherwise obtain. That condition could not be constructed here.

Benchmark validity. Five of the fifteen run-to-run flips are grading artefacts, and `miss_func` substantially measures tool-list perception rather than reasoning (Section 4.1).

7 Conclusion

On this benchmark and this model, *where* reflection sits in the loop proved less informative than *what it is told*: even a substantially richer input did not change how many failures were recovered, only which ones. The number is capped by a small pool of borderline tasks any retry can reach, and reflection’s observable effect is to shift the agent’s act/ask threshold, helping in one category as much as it hurts in another.

The methodological lesson generalises: when the system under test is not reproducible and re-attempting is part of the architecture, an uninformative control arm is necessary rather than optional. Measured against its own first attempt, Reflexion gains 7.5 pp at $p < 10^{-4}$. Measured against a sham that says nothing, it gains nothing. Only the second comparison controls for re-attempting.

References

- [1] M. Kamilov. Reasoning–action loop architectures on BFCL v3: experiment code, prompts, trajectory logs and per-task results. <https://github.com/mirodilkamilov/seminar>, 2026.
- [2] S. Kapoor, B. Stroebel, Z. S. Siegel, N. Nadgir, and A. Narayanan. AI agents that matter. arXiv:2407.01502, 2024.
- [3] Q. McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.
- [4] S. G. Patil, H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stoica, and J. E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR, 2025.
- [5] Qwen Team. Qwen3 technical report. arXiv:2505.09388, 2025.
- [6] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. arXiv:2303.11366.
- [7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.
- [8] Q. Zeng, S. Rajkumar, D. Wang, N. Gyanchandani, and W. Yan. Reflect before act: Proactive error correction in language models, 2025. arxiv:2509.18607.

Appendix: prompts and signals

Every prompt below is reproduced verbatim from the committed source [1]. Braces such as {signal} are substitution placeholders filled at run time. Together these make the report’s central claim checkable: at each rung of the ladder the arms differ in exactly one place, and that place is printed here.

A.1 Shared system prompt core (Baseline)

Used unchanged by *every* arm. The Baseline’s system prompt is exactly this text and nothing else; other arms append a scaffold to it.

```
You are an agent that completes user tasks by calling the provided tools. Call tools whenever they are needed
to satisfy the user's request.
If none of the available tools can fulfil the request, say so instead of guessing. If the request is missing
a value that a tool requires, ask the user for it.
Keep calling tools until the current request is fully handled. When you have nothing left to call, reply in
natural language summarising what you did - a message with no tool call ends the current turn.
```

A.2 ReAct scaffold

Appended to the core above, after a blank line. This is the entire ReAct manipulation.

```
Work in an explicit reason-act loop. Every time you call a tool, begin that SAME message with the literal
label 'Thought:' followed by one or two sentences - what you know, what you still need, and which tool to
call next - then make the tool call(s) in that same message.
A Thought with no tool call attached ends the current turn. After each tool result, any further tool call
again begins with a 'Thought:' label in that same message.
```

A.3 The reflection prompt, and the state variant

Reflexion’s reflection call. It is a single LLM call with no tools, at the same temperature and token budget as the action loop.

```
Below is the full transcript of your previous attempt at a multi-turn tool-calling task. The attempt was
graded as FAILED.

{attempt_text}

Grader failure signal: {signal}

Reflect on this failed attempt. Reply in exactly this format:
What I tried: <one or two sentences>
What went wrong: <one or two sentences - your best diagnosis, using the failure signal>
Lesson: <one or two sentences of concrete, actionable guidance>

Whoever reads this reflection later will not see this transcript - only the reflection itself. Make the
Lesson self-contained.
```

Reflexion+State uses a prompt that is **byte-identical to the above** except for three inserted lines after the failure signal (551 characters against 649). This is the whole of the contribution:

```
+
+For reference, here is how your own actions changed the environment state, turn by turn:
+{state}
```

A.4 The two sham reflections

The Sham arm’s lesson slot holds this fixed constant. It is written by no one and contains nothing about the failure, but it is form-matched to Appendix A.3’s output: the same three

headings, comparable length.

```
What I tried: I attempted the task using the tools provided.
What went wrong: Some part of my approach did not match what the task required.
Lesson: I should re-read the request carefully, verify preconditions before acting, and double-check tool arguments.
```

Sham-lite is identical except that the Lesson line drops its two imperative clauses, giving "Lesson: I should re-read the request carefully." That is the entire difference between the two control arms: 257 characters against 188, and they score 103 and 102 out of 200.

A.5 The retry preamble

The user message every retry opens with, before turn 0. The template is **identical in all four retry arms**; only the reflection text differs. One signal/reflection pair is appended per failed attempt. Example, attempt 2 of a Sham run:

```
Note: a previous, separate attempt at this task failed.
Attempt 1 failure signal: After your attempt, the TicketAPI environment was left in a state that does not match what the task required.
Attempt 1 reflection: What I tried: I attempted the task using the tools provided.
What went wrong: Some part of my approach did not match what the task required.
Lesson: I should re-read the request carefully, verify preconditions before acting, and double-check tool arguments.
The task now restarts from the beginning.
```

A.6 Sanitized failure signals

The grader's raw verdict leaks the expected answer, so it is mapped to one of four templates carrying only an environment class or a turn index. These are the complete set; no value from the ground truth ever reaches the agent.

```
[state gate]      After your attempt, the {ClassName} environment was left in a state that does not match what the task required.

[response gate]   By the end of user turn {n} (counting from 1), the output of one or more required tool calls was missing: a call the task required was never made, or was made with different arguments.

[empty turn]      You ended user turn {n} (counting from 1) without making any tool calls, but that turn required action.

[irrelevance]     You made tool calls in user turn {n} (counting from 1), but the correct behavior there was to make none (e.g. ask the user for missing information, or say that no available tool fits).
```

The state gate carries no turn index: the checker does not record which turn produced the mismatch, so those signals are class-level only.

A.7 An example state timeline

The block Reflexion+State inserts at {state}, here the real one from miss_param_32 (Section 5): it shows the agent the file it actually wrote, which let it correct 2.0000 to 2.0. Turns changing nothing are listed as such — the under-acting signal.

```
Turn 1: no state changes
Turn 2: no state changes
Turn 3:
  GorillaFileSystem:
    root = "<Directory: project, Parent: None, Contents: {'Spring2023Draft': <<File: Spring2023Draft, Content : These are the notes for Spring 2023.>>, 'PastSeasons': <Directory: PastSeasons, Parent: project, Contents: {}>, 'result.txt': <<File: result.txt, Content: The logarithm of 36 to base 6 is: 2.0>>>>"
```